

Report of the project of Security and Software Application

Luca Sforza

Sapienza

sforza.2050030@studenti.uniroma1.it

Abstract—Questo report descrive il lavoro svolto per il progetto d’esame del corso di Security and Software Application, Sapienza uniroma1.

Il lavoro introduce la correzione degli smart contract dati insieme alla traccia del progetto.

E’ stato utilizzato un approccio Agile, ogni parte del progetto è stata suddivisa in un iterazione della metodologia Agile.

I. INTRODUCTION

L’obiettivo di questo progetto è analizzare e correggere gli smart contract forniti nella traccia dell’esame del corso Security and Software Application della Sapienza. Il lavoro si concentra sull’individuazione delle vulnerabilità presenti nel codice,

L’obiettivo di questo progetto è analizzare e correggere gli smart contract forniti nella traccia dell’esame del corso Security and Software Application della Sapienza. Il lavoro si concentra sull’individuazione delle vulnerabilità presenti nel codice tramite la definizione delle invarianti descritte nelle specifiche del progetto.

La parte di Analisi è stata fatta usando un approccio Model Driven basato su UML (Unified Model Language) particolarmente adatto per la modellazione di sistemi Solidity.

II. PULIZIA DEL CODICE INIZIALE

Il codice che è stato fornito era in un pessimo stato.

La visibilità degli attributi è quella di default (TODO metti cosè) e il codice è mal indentato non seguendo nessuno standard di formattazione.

Per visualizzare tutti questi problemi è possibile utilizzare dei linter.

A. Ambienti di sviluppo Solidity

Per questo progetto è stato utilizzato l’ambiente di sviluppo Solidity foundry che fornisce 3 tool principali per lo sviluppo.

- forge
- anvil
- cast

B. Problemi logici

Oltre a questi problemi di gestione del codice nel codice fornito esistono vari problemi logici.

Sono stati trovati tramite un approccio Model Driven.

Usando UML (Unified Model Language) è possibile in un modo molto naturale a convertirlo in codice Solidity. Nella letteratura si propongono Meta-Modelli.

Il diagramma delle classi UML mostrato in figura definisce i vincoli come invarianti del sistema, che devono essere sempre mantenuti veri durante l’esecuzione del codice Solidity.

Inanzitutto l’età di un contribuente è salvata come un intero che deve essere incrementato ogni anno.

Un approccio migliore è quello di salvarsi il timestamp del blocco in cui è salvata eseguita la transazione.

Ogni metodo che definiamo in un contratto Solidity vengono chiamate come transazioni verso il contratto stesso. In Solidity abbiamo accettato a tutte le informazioni di questa transazione (address del chiamante, ether inviati, timestamp del blocco, ecc...) tramite un costrutto del linguaggio stesso, la keyword: msg

Quindi l’età adesso è calcolabile, infatti diventa un metodo del contratto

Si possono usare approcci di verifica formale del Software per dimostrare matematicamente la correttezza del codice fornito.

Però per questo progetto è stato utilizzato un altro approccio, il fuzzy testing.

Molti dei vincoli imposti dall’UML possono essere vincolati nel codice direttamente grazie alla sintassi del linguaggio (le associazioni, le classi, le interfacce, ecc...).

Invece altri vincoli (specialmente i vincoli esterni definiti) devono essere definiti come metodi che analizzano lo stato del contratto per verificare che i vincoli siano veri.

III. PRIMO ESERCIZIO

Nel primo esercizio viene richiesto di verificare il vincolo degli sposi.

Più formalmente si deve verificare che la formula in logica del primo ordine:

$$\forall a \text{ Taxpayer}(a) \wedge \forall b \text{ spouse}(a, b) \implies \text{spouse}(b, a) \quad (1)$$

Quindi per ogni Taxpayer a , il suo sposo deve essere sposato al taxpayer a .

Questo vincolo non può essere modellato attraverso la sintassi del linguaggio Solidity, come invece è possibile con le invarianti definite dalla formula ϕ generata dal diagramma UML delle classi.

Quindi va verificato che durante l’esecuzione dello smart contract questo vincolo venga mantenuto in ogni possibile stato.

Sfortunatamente questo problema è indecidibile ,però possiamo optare al fuzzy testing.

Il vincolo può essere facilmente scritto in Solidity nel seguente modo:

IV. ESERCIZIO 2

$$\forall a, b \quad (2)$$